39 MONGODB RELATIONSHIPS — SIMPLE VERSION

theory/38 covers this in full depth. This is the short, plain-language version — read this first if 38 felt like too much at once.

THE BASIC QUESTION

MongoDB stores data as separate documents in collections (like separate Student documents in a "students" collection). A "relationship" is just: how do two related documents know about each other? There are only two ways to do it.

WAY 1 — EMBEDDING (put the data INSIDE the document)

```
{
  name: "Rahul",
  address: { city: "Pune", street: "MG Road" }
}
```

The address lives right inside the student. One read gets you everything.

Use this when the extra data:

- is small
- always belongs to just ONE student
- is always read together with the student (never on its own)

WAY 2 — REFERENCING (store just an ID, not the data)

```
// students collection
{ name: "Rahul", course: "65f0a1..." }   // just an id, like a pointer

// courses collection
{ _id: "65f0a1...", title: "CS", credits: 4 }
```

The student only stores the Course's _id. The actual course data lives in its own collection.

Use this when the extra data:

- is shared by MANY students (100 students, 1 "CS" course — no point copying the same course info 100 times)
- has its own independent life (a course exists even with zero students enrolled)
- can grow very large over time

HOW YOU GET THE REAL DATA BACK — populate()

A reference alone just gives you an id, not the actual course info. `.populate()` is the command that goes and fetches it for you:

```
this.studentModel.findById(id).populate('course').exec();
```

Before populate: course: "65f0a1..." After populate: course: { title: "CS", credits: 4 }

Think of populate() as "go look up what this id actually points to."

THE THREE RELATIONSHIP SHAPES

ONE-TO-ONE — one student, one address → usually embed ONE-TO-MANY — one course, many students in it → reference (course

```
                stores nothing, OR stores an array of student ids)
MANY-TO-MANY  — many students, many courses each -> reference, each
                side stores an array of the other side's ids
```

THE ONE THING TO REMEMBER

MongoDB does NOT check that a reference is valid. You can store a course id that doesn't exist, and nothing stops you. Unlike SQL's FOREIGN KEY, this is 100% your responsibility to get right in code.

★ RULE OF THUMB

Small + always-together + belongs to one parent → embed it. Shared + independent + grows big → reference it, then populate() when you actually need the real data.

See also: **theory/38-mongodb-relationships.txt** (full version with code for every case), **theory/31-mongoose-schema-decorators.txt**